26W CST8276 Section 010 Advanced Database Topics

Report

GPS location mapping using Google APIs (map with location over time and ‘route’)

Group 2 (CST8276 010 – 2)

Prepared By <order by last name>:

Barbesin, Luca **<barb0285@algonquinlive.com>**
Cheng, Annabel **<chen1036@algonquinlive.com>**
Derdar, Sarra **<derd0004@algonquinlive.com>**
Gadhvi, Ambika **<gadh0015@algonquinlive.com>**

Lab Professor: Lurie, Abbas

Algonquin College Professor: Dr. Sarfraz Khan

Date of Submission: 2026-03-01

Group 2

Table of Contents

Contents

Table of Contents 1
List of Tables, Images, and Figures 2
Introduction 3
Project Description 3
1. Who: who cares and who does it affect 3
2. What: what is the data aspect of the issue 4
3. Where: is this a client-side problem, a replication problem, a server capacity problem, etc. 4
4. When: any timing aspects (processes, steps, etc.) 5
5. Why: any laws, regulations, or other constraints 5
6. How: how does the issue happen and get resolved 6
Solution Description & Results 6
1. What you did for the solution 6
1.1. Components, scripts, database setup, etc. 6
1.2. Screen Shots Details 8
1.3. Summary of Results 15
Work Plan 16
1. Table with components and deliverables 16
Lesson Learned 17
1. Risks that needed to be mitigated (look at your list of known risks) 17
2. Overall Experience 18
References 19

Group 2

List of Tables, Images, and Figures

Table 1: REST API Endpoints 7
Table 2: Project Files 8
Table 3: Table with components & deliverables 16
Table 4: Risk Management Scenarios 17

Image 1: Oracle Schema Creation 9
Image 2: GPS Waypoints Data in Oracle 9
Image 3: Index Performance with EXPLAIN PLAN 10
Image 4: Flask Application Running 11
Image 5: API Response in Browser 11
Image 6: Application Home Page with Dropdowns 12
Image 7: Route Drawn on Google Maps 13
Image 8: Route Animation in Progress 14
Image 9: Waypoint List in Sidebar 14
Image 10: Multiple Routes Comparison 15

Group 2

Introduction

GPS location mapping is widely used in modern applications such as navigation, delivery tracking, and location-based services. This project focuses on GPS location mapping using Google APIs, specifically visualizing location data over time and displaying routes on an interactive map. The project examines how GPS data is collected, stored in a relational database, processed through backend services, and rendered on the client side, with emphasis on database design, querying efficiency, and data visualization.

This topic was chosen because it directly relates to key concepts in advanced database systems and computer programming. Working with GPS data provides practical experience with time-series and geospatial data, API integration, and performance considerations [4]. Additionally, using Google Maps APIs exposes students to industry-standard tools while highlighting important issues such as data accuracy, security, and privacy. Overall, the project offers a relevant and applied learning opportunity that bridges theoretical database concepts with real-world software development.

Project Description

1. Who: who cares and who does it affect

This project impacts everyone associated with this application of location-based data. This includes the people whose location is being tracked as well as the people using this map or route view to make decisions.

(a) End users that are tracked: These are the users of the application, or the users that are being tracked. They are impacted because this application of location history is personal and can reveal their habits, such as their residence, workplace, and more.
(b) Organizations that utilize this application of tracking: They are the groups of people that are utilizing this application of location history. They are impacted because it will allow them to more efficiently dispatch their employees, provide proof of service, etc.
(c) Clients of these organizations: These are the clients that are waiting on the services of the above-mentioned groups. They are impacted because it will allow them to have more accurate estimations of time arrival.
(d) IT/Developers/Data Admins: They are impacted because the database will need to handle large amounts of GPS data in an efficient way.
(e) /Privacy Admins/Legal: They are impacted because of the need for privacy with this application of location history.

(f) Google (API provider): They are impacted because of the need for them to provide this service in a way that it meets their terms of service.

2. What: what is the data aspect of the issue

The data part of GPS location mapping refers to the data process involved in the collection, storage, cleaning, storage, cleaning, and querying of the points to ensure that they are converts into routes and useful information that can be represented in map.

(a) Data collected: The GPS points are essentially data points that have information such as latitude, longitude, time, accuracy, speed, direction, altitude, device id, session id, and user id.
(b) Derivation of usefulness: The GPS points are converted into useful information that the end user wants to see, such as routes, paths, distance, time, speed, stops and segments.
(c) Databases requirements and issues:
(d) The GPS points are essentially a database problem; they are time series data and have spatial component. The challenges in the GPS points are:

- High volume: The data points are in thousands and are generated in rapid succession.
- Query performance: The points for a specific user during a specific time or points near a specific point must be retrieved.
- Indexing: The data points have to be indexed, not just by time, but also by the spatial component.
- Data Quality: The accuracy of GPS points is a challenge, and there are problems such as drift, duplication, and time stamps [3].
- Security: The GPS points are related to the user's location, and therefore there are security challenges.

3. Where: is this a client-side problem, a replication problem, a server capacity problem, etc.

This is primarily a client-side visualization challenge with backend database storage. The system must efficiently sync data between backend and frontend. The problem spans:

(a) Client-side: Browser-based map rendering, route visualization, and user interface
(b) Server-side: API endpoints for data retrieval and processing
(c) Database: Efficient storage and indexing of geospatial data
(d) Integration layer: Communication between frontend visualization and backend data sources
(e) Mobile considerations: Potential for responsive design to support mobile viewing

(f) The solution must address the full stack, from database design to front-end rendering, with particular attention to the performance challenges of rendering potentially complex routes on a map interface.

This problem exists across multiple layers of a modern software ecosystem. At the data layer, GPS information must be stored in a structured relational database capable of handling spatial queries. At the server layer, APIs are required to manage requests, retrieve filtered value sets, and provide clean, formatted coordinate sequences to the client. On the client side, the visualization layer must interpret this data and convert it into dynamic, interactive elements using Google Maps. The system must also operate across different devices such as laptops, tablets, and mobile phones, each with varying processing capabilities and map rendering performance. These multiple "locations" in which the problem exists highlight the need for a solution that is scalable, modular, and adaptable.

4. When: any timing aspects (processes, steps, etc.)

Timing plays an important role in GPS location mapping because the system must support both real-time tracking and historical data analysis. In real-time scenarios, location data needs to be captured, processed, and displayed on the map as it changes, such as during live delivery or vehicle tracking. In historical use cases, previously stored GPS data must be retrieved efficiently to review routes, travel patterns, and movement over specific time periods.

Additional timing considerations include filtering location data by date and time, visually representing movement over time on the map, and ensuring fast loading and rendering of routes. The system must also balance data freshness with overall performance, especially when handling frequent updates or large datasets. As a result, the solution must be capable of handling both immediate, real-time data processing and accurate, responsive access to historical GPS information.

5. Why: any laws, regulations, or other constraints

GPS location mapping involves collecting and using personal location data, which is considered sensitive information. Because of this, there are legal and regulatory requirements that must be followed. In Canada, the Personal Information Protection and Electronic Documents Act (PIPEDA) [5] require organizations to obtain clear user consent before collecting location data, limit data collection to only what is necessary, and ensure that all location information is stored securely.

In addition to legal requirements, this project is also constrained by technical and platform-related rules. Google Maps APIs are governed by Google's Terms of Service, which include usage

limits, request quotas, and restrictions on how location data can be stored or reused [2]. These rules also prohibit tracking users without explicit permission. Together, these laws and constraints influence how the system is designed, how data is handled, and how GPS information is processed and displayed [5].

6. How: how does the issue happen and get resolved

The issues in GPS location mapping primarily arise from the characteristics of GPS data and the way it is stored and processed within the database. GPS data is generated at high frequency and may contain inaccurate or incomplete records due to signal loss, resulting in missing points or spatial drift in reconstructed routes [3]. Inconsistent or unsynchronized timestamps can also lead to incorrect temporal ordering of records, which directly affects route generation and time-based queries. From a system perspective, improperly managed Google Maps API keys introduce security and billing risks, while irregular data ingestion rates can cause fragmented or uneven route visualization when querying location data.

These issues are addressed through careful database design and controlled data processing. GPS records are stored with explicit timestamp and accuracy attributes, allowing low-quality data to be filtered during queries. Server-side time normalization ensures consistent temporal ordering of records before route construction. Indexing strategies on timestamp and spatial attributes improve query performance for route reconstruction. API keys are secured using application and usage restrictions to prevent unauthorized access. Additionally, data filtering and smoothing techniques are applied at the query or processing layer to reduce noise and produce stable, continuous route representations.

Solution Description & Results

1. What you did for the solution

Our group designed and implemented a complete GPS location mapping system that stores, retrieves, and visualizes route data using Oracle Database 26ai. The solution demonstrates the full lifecycle of GPS data from structured storage in a relational database, through a Python REST API, to interactive rendering on Google Maps. The system was built in three tiers: a database layer, a backend application layer, and a browser-based frontend.

1.1. Components, scripts, database setup, etc.

The solution is composed of the following components:

1.1.1. Backend Layer — Python 3 with Flask and python-oracledb

The backend is a lightweight Flask application [4] (app.py) that connects to Oracle 26ai [1] using the python-oracledb driver in thin mode, no Oracle client installation is required. The application exposes four REST API endpoints:

Table 1: REST API Endpoints

Endpoint	Description
GET /api/users	Returns all users from GPS_USERS ordered by username
GET /api/sessions?user_id=N	Returns all sessions for a given user, joined with GPS_ROUTES for summary statistics
GET /api/waypoints?session_id=N	Returns all waypoints for a session ordered by recorded_at, using the composite index
GET /api/summary?session_id=N	Returns the computed route summary — distance, duration, and waypoint count

All database queries use parameterised inputs cast to integers before execution to prevent injection. Timestamps are returned as Python datetime objects and formatted in Python using strftime, avoiding the need for Oracle TO_CHAR formatting strings in SQL. The Flask server runs in debug mode on port 5000 and serves the static frontend from the /static directory.

1.1.2. Frontend Layer — HTML, CSS, and JavaScript with Google Maps API

The frontend is a single HTML file (static/index.html) that loads entirely in the browser. It communicates with the Flask backend using the browser's native fetch() API. The Google Maps JavaScript API [2] is loaded asynchronously and initialised with Ottawa, Ontario as the default map centre.

The interface is divided into two panels. The left sidebar displays route statistics: distance in kilometres, duration in minutes, waypoint count, session label, and start/end times, along with a scrollable list of all waypoints with their coordinates and timestamps. The right panel is the full-height Google Maps canvas.

The frontend supports the following interactions:

- User selection dropdown — populates from /api/users on page load
- Session selection dropdown — populates from /api/sessions when a user is chosen

- Show Route button — fetches waypoints and summary, draws a blue polyline on the map with green start marker and red end marker, fits the map bounds to the route
- Animate Route button — moves a yellow arrow marker along the route waypoint by waypoint at 400ms intervals, updating the status bar with the current timestamp
- Clear button — removes all map overlays and resets the sidebar

1.1.3. Project Files

Table 2: Project Files

File	Purpose
schema.sql	Creates gpsuser account and all four database tables with index
seed_data.sql	Inserts 2 users, 3 sessions, 37 waypoints, 3 route summaries
seed_data_extra.sql	Adds 2 more users and 9 additional sessions for all group members
app.py	Flask backend — DB connection, 4 REST endpoints, static file serving
requirements.txt	Python dependencies: flask, oracledb
static/index.html	Complete frontend: Google Maps, dropdowns, animation, stats sidebar

1.2. Screen Shots Details

The following screenshots document the complete working solution, from database setup through to the live map demonstration.

Screenshot 1: Oracle Schema Creation

This screenshot shows the successful execution of schema.sql in SQL*Plus, confirming that the gpsuser account and all four tables were created correctly in Oracle 26ai [1].

```
AC SYS@localhost:1521/FREEPDB1) > @C:\Users\c-ann\Desktop\26W_CST8276_AdvancedDB\GPS-Route-Tracker\schema.sql

User created.

Grant succeeded.

Grant succeeded.

Table created.

Table created.

Table created.

Index created.

Table created.

Commit complete.
```

Image 1: Oracle Schema Creation

Screenshot 2: GPS Waypoints Data in Oracle

This screenshot demonstrates that the GPS data is correctly stored in the Oracle database, showing real latitude/longitude/timestamp records.

```
AC GPSUSER@localhost:1521/FREEPDB1) > SELECT waypoint_id, session_id, latitude, longitude, recorded_at
  2  FROM gps_waypoints
  3  WHERE session_id = 1
  4  ORDER BY recorded_at;

WAYPOINT_ID SESSION_ID   LATITUDE  LONGITUDE RECORDED_AT
___________ __________ __________ __________ ____________________________

          1          1    45.4235   -75.6979 20-MAR-26 08.00.00.000000 AM
          2          1    45.4218   -75.6962 20-MAR-26 08.03.00.000000 AM
          3          1      45.42   -75.6945 20-MAR-26 08.06.00.000000 AM
          4          1    45.4182   -75.6928 20-MAR-26 08.09.00.000000 AM
          5          1    45.4165   -75.6912 20-MAR-26 08.12.00.000000 AM
          6          1    45.4148   -75.6896 20-MAR-26 08.15.00.000000 AM
          7          1     45.413   -75.6882 20-MAR-26 08.18.00.000000 AM
          8          1    45.4112    -75.687 20-MAR-26 08.21.00.000000 AM
          9          1    45.4094   -75.6858 20-MAR-26 08.24.00.000000 AM
         10          1    45.4076   -75.6848 20-MAR-26 08.27.00.000000 AM
         11          1    45.4058    -75.684 20-MAR-26 08.30.00.000000 AM
         12          1     45.404   -75.6833 20-MAR-26 08.33.00.000000 AM
         13          1    45.4022   -75.6828 20-MAR-26 08.36.00.000000 AM
         14          1    45.4003   -75.6824 20-MAR-26 08.39.00.000000 AM
         15          1    45.3985    -75.682 20-MAR-26 08.45.00.000000 AM

15 rows selected.
```

Image 2: GPS Waypoints Data in Oracle

Screenshot 3: Index Performance with EXPLAIN PLAN

This screenshot demonstrates a key database design decision: the composite index on GPS_WAYPOINTS(session_id, recorded_at). The EXPLAIN PLAN shows that Oracle uses the index rather than a full table scan, which would be critical for performance at scale with millions of GPS records.

```
AC GPSUSER@localhost:1521/FREEPDB1) > EXPLAIN PLAN FOR
  2  SELECT latitude, longitude, recorded_at
  3  FROM gps_waypoints
  4  WHERE session_id = 1
  5  ORDER BY recorded_at;

Explained.

AC GPSUSER@localhost:1521/FREEPDB1) > SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
Plan hash value: 371890401

--------------------------------------------------------------------------------
| Id  | Operation                   | Name                | Rows  | Bytes | Cost (%CPU)| Time     |
--------------------------------------------------------------------------------
|   0 | SELECT STATEMENT            |                     |    15 |   780 |     1   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID| GPS_WAYPOINTS       |    15 |   780 |     1   (0)| 00:00:01 |
|*  2 |   INDEX RANGE SCAN          | IDX_WP_SESSION_TIME |    15 |       |     1   (0)| 00:00:01 |
--------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("SESSION_ID"=1)

Note
-----
   - dynamic statistics used: dynamic sampling (level=2)

18 rows selected.
```

Image 3: Index Performance with EXPLAIN PLAN

Screenshot 4: Flask Application Running

This screenshot confirms the Flask backend is successfully started and connected to Oracle 26ai [1], ready to serve API requests.

```
(.venv) PS C:\Users\c-ann\Desktop\26W_CST8276_AdvancedDB\GPS-Route-Tracker> python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 388-966-759
```

Image 4: Flask Application Running

Screenshot 5: API Response in Browser

This screenshot demonstrates the REST API returning GPS data from Oracle as JSON, confirming the full backend-to-database connection is working.

localhost:5000/api/waypoints?ses

localhost:5000/api/waypoints?session_id=1 Summarize

Pretty-print


```
[
  {
    "accuracy": 5.0,
    "altitude": 70.2,
    "lat": 45.4235,
    "lng": -75.6979,
    "recorded_at": "2026-03-20 08:00:00",
    "speed": 4.5
  },
  {
    "accuracy": 4.5,
    "altitude": 70.1,
    "lat": 45.4218,
    "lng": -75.6962,
    "recorded_at": "2026-03-20 08:03:00",
    "speed": 4.8
  },
  {
    "accuracy": 4.0,
    "altitude": 69.8,
    "lat": 45.42,
    "lng": -75.6945,
    "recorded_at": "2026-03-20 08:06:00",
    "speed": 5.1
  },
  {
    "accuracy": 4.2,
    "altitude": 69.5,
    "lat": 45.4182,
    "lng": -75.6928,
    "recorded_at": "2026-03-20 08:09:00",
    "speed": 4.9
  },
  {
    "accuracy": 4.8,
    "altitude": 69.2,
    "lat": 45.4165,
    "lng": -75.6912,
    "recorded_at": "2026-03-20 08:12:00",
    "speed": 5.0
  },
```


Image 5: API Response in Browser

Screenshot 6: Application Home Page with Dropdowns

This screenshot shows the loaded web application with the user and session dropdown menus populated from the Oracle database, confirming the full frontend-to-backend-to-database chain is working.

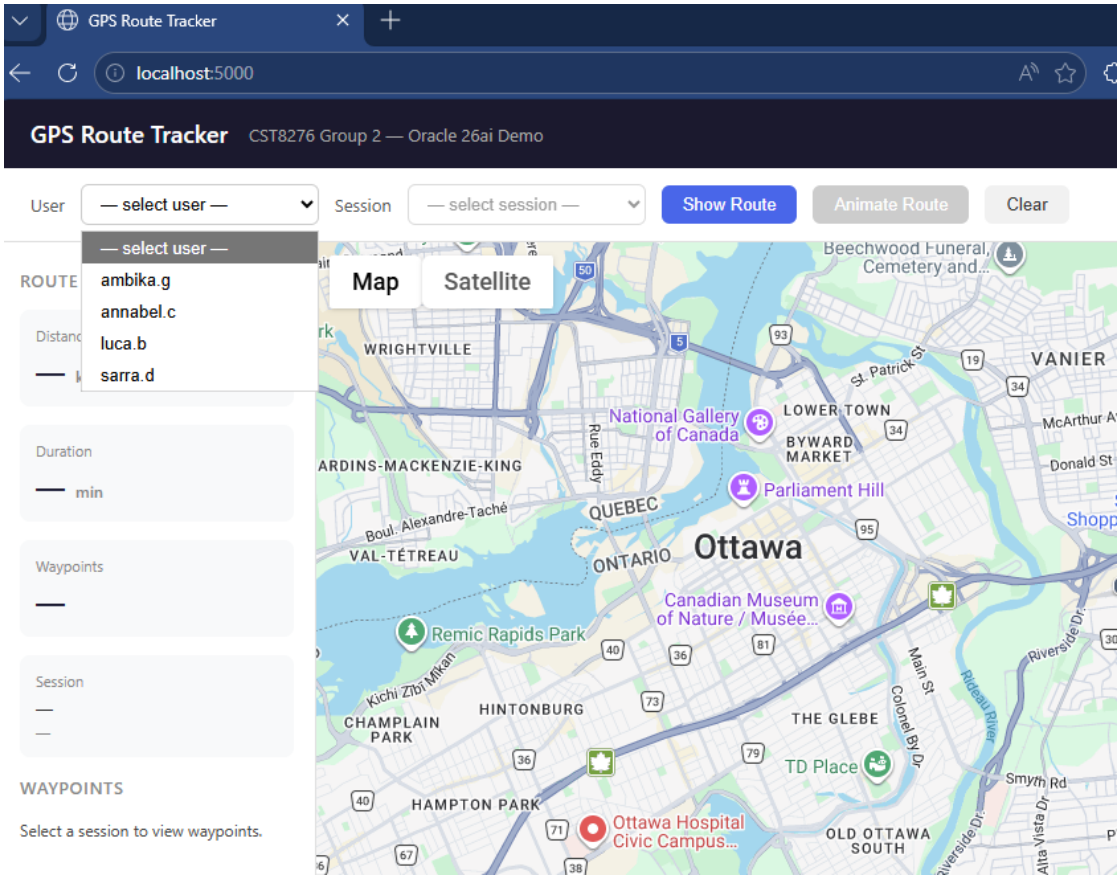

Image 6: Application Home Page with Dropdowns

Screenshot 7: Route Drawn on Google Maps

This is the **primary demonstration** screenshot. It shows a GPS route rendered as a blue polyline on Google Maps, with a green marker at the start and a red marker at the end. The left sidebar displays the computed statistics for the selected route, distance, duration, and waypoint count, all retrieved from Oracle.

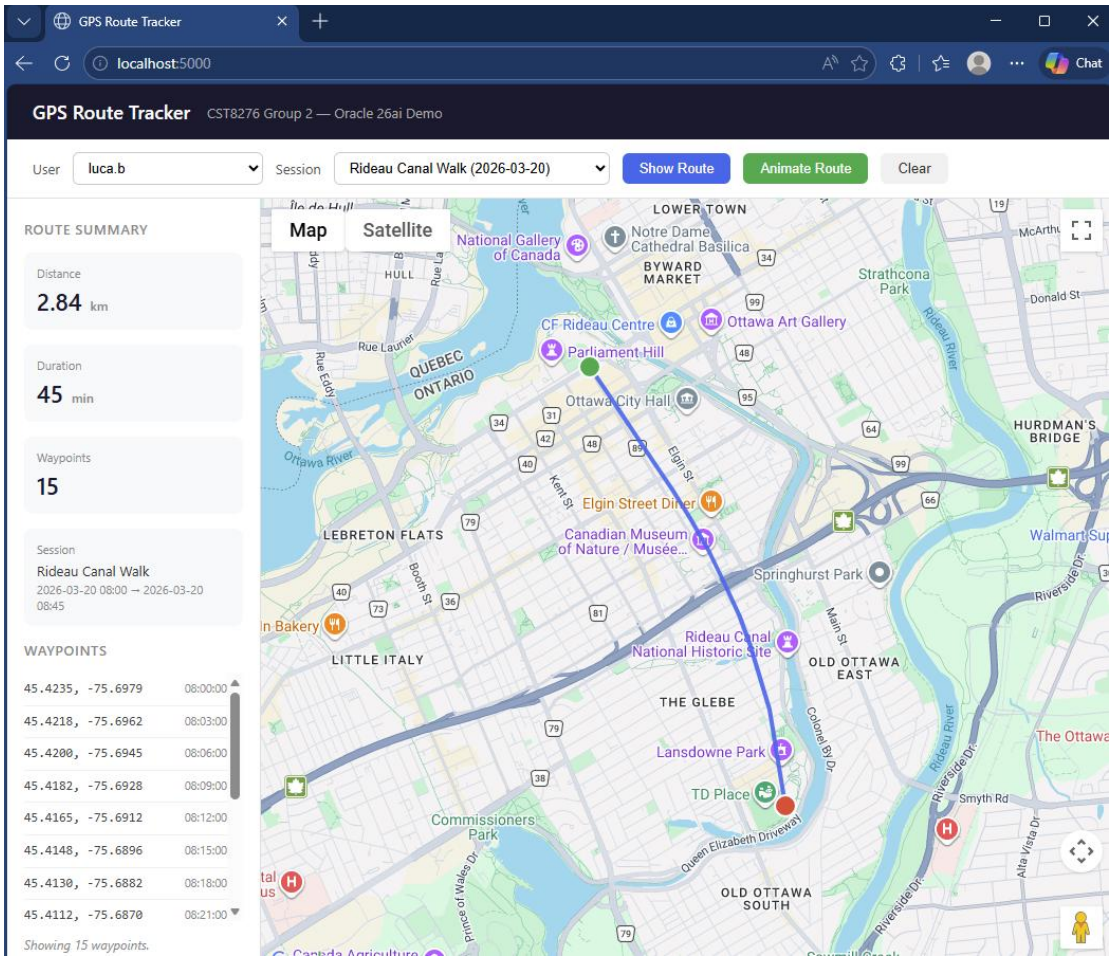

Image 7: Route Drawn on Google Maps

Screenshot 8: Route Animation in Progress

This screenshot captures the route animation feature: a yellow arrow marker moving along the route waypoint by waypoint. The status bar at the bottom of the sidebar shows the current timestamp being replayed, demonstrating time-series playback of historical GPS data.

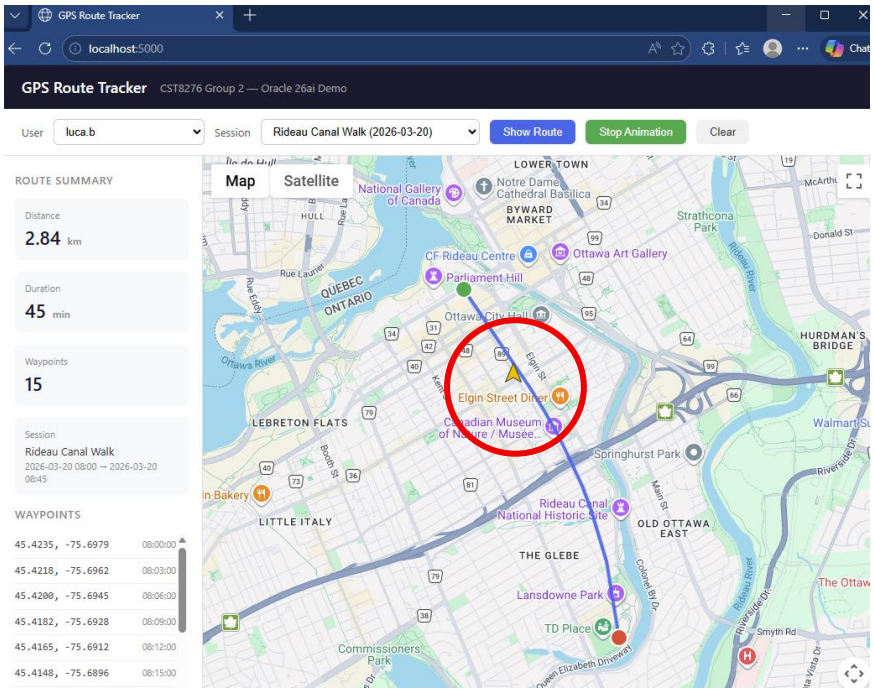

Image 8: Route Animation in Progress

Screenshot 9: Waypoint List in Sidebar

This screenshot shows the waypoint list panel in the sidebar, displaying the coordinates and timestamps of each recorded GPS point for the selected session. This demonstrates that the application is retrieving and displaying the complete time-ordered dataset from Oracle.

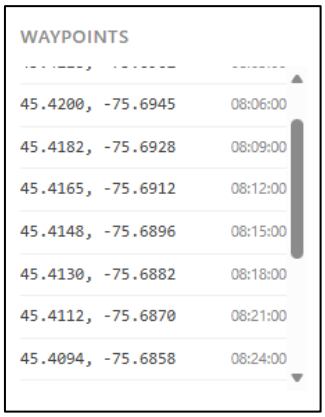

Image 9: Waypoint List in Sidebar

Screenshot 10: Multiple Routes Comparison

This final screenshot demonstrates switching between sessions for different users, showing how the map updates to display a different route. This confirms the system correctly isolates GPS data by session and user, a key requirement of the database design.

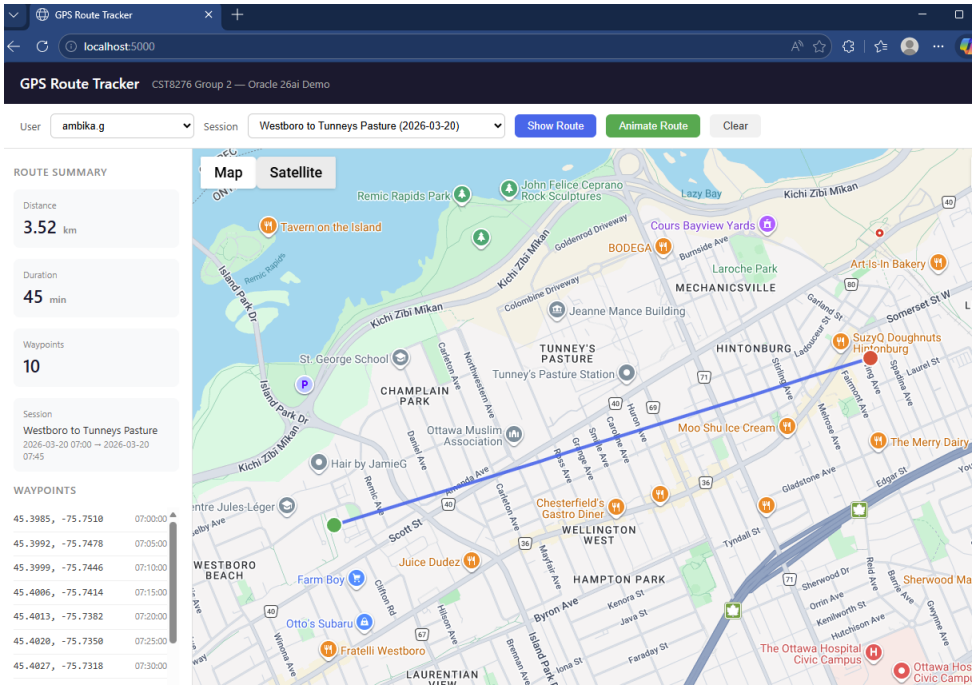

Image 10: Multiple Routes Comparison

1.3. Summary of Results

The completed solution successfully demonstrates GPS location mapping using Oracle 26ai [1] and the Google Maps JavaScript API [2]. All planned components were implemented and are fully functional. The system correctly stores GPS data in a normalized relational schema, retrieves it efficiently using indexed queries, exposes it through a REST API, and renders it as interactive routes on a live map.

The composite index on GPS_WAYPOINTS(session_id, recorded_at) was confirmed to improve query performance through EXPLAIN PLAN analysis. The animation feature demonstrates time-series data playback, a meaningful application of the timestamp ordering built into the database design. All four group members are represented in the sample dataset with real Ottawa GPS routes, giving the demonstration variety and geographic relevance.

Group 2

Work Plan

1. Table with components and deliverables

Table 3: Table with components & deliverables

Component / Deliverable	Planned Timeline	Planned Hours	Actual Timeling	Actual Hours	Responsible Member(s)
1. Initial Proposal					
1.1 Draft Proposal, Introduction & Work Plan	Week 3	5 hrs	Week 3-4	6 hrs	Annabel
1.2 Who & What Sections	Week 4	2 hrs	Week 4	2 hrs	Esakor
1.3 Where & When Sections	Week 4	3 hrs	Week 4	3 hrs	Sarra
1.4 Why & How Sections	Week 4	2 hrs	Week 4	2 hrs	Ambika
1.5 Solution Demonstration Section	Week 4	5 hrs	Week 4	5 hrs	Luca
2. Final Proposal					
2.1 Review, Final Editing & Formatting	Week 5-6	2 hrs	Week6	2 hrs	Annabel
2.2 Expanded Background & Context	Week 5-6	0.5 hrs	Week6	0.5hr	Esakor
2.3 Frontend Development & Risk Update	Week 5-6	0.5 hrs	Week6	0.5hr	Luca
2.4 Unit Testing & Gap Analysis	Week 5-6	0.5 hrs	Week6	0.5hr	Ambika
2.5 Refined Demonstration Plan	Week 5-6	0.5 hrs	Week6	0.5hr	Sarra
3. Final Report					
3.1 Draft, Introduction, Work Plan, Lessons Learned, Final Editing & Formatting	Week 8-9	8 hrs	Week 11	2 hrs	Annabel
3.2 Problem Description (Who & What)	Week 8-9	5 hrs	Week 11	1 hr	Sarra
3.3 Problem Description (Where & When)	Week 8-9	5 hrs	Week 11	1 hr	Luca
3.4 Problem Description (Why & How)	Week 8-9	5 hrs	Week 11	1 hr	Ambika
3.5 Solution Description & Results	Week 8-9	6 hrs	Week 11	8 hrs	Annabel
4. Presentation					
4.1 Presentation Outline	Week 10	3 hrs	Week 12	2 hrs	Annabel
4.2 Background & Context Slides	Week 10	6 hrs	Week 12	3 hrs	Sarra

4.3 Demonstration Slides	Week 10	6 hrs	Week 12	2 hrs	Ambika
4.4 Visual Design & Timing	Week 10	6 hrs	Week 12	2 hrs	Ambika
4.5 Final Review & Practice/ Presentation	Week 10	6 hrs	Week 12	2 hrs	Luca

Lesson Learned

1. Risks that needed to be mitigated (look at your list of known risks)

At the proposal stage, our group identified five risks that could affect the project. Looking back, all five materialised to some degree during development. The table below maps each original risk to what actually happened and how the group responded.

Table 4: Risk Management Scenarios

Risk	What Actually Happened	How we handled it
API Dependency Risk: Google Maps usage limits or key misconfiguration could break map functionality	The Maps JavaScript API [2] loaded correctly once the API key was restricted to localhost:5000. No quota issues were hit during development.	We restricted the API key to our local development URL immediately after creation. This prevented unauthorised use and kept us within the free tier limits.
Environment Configuration Risk: Different OS and Python versions across team members could cause setup issues	One team member encountered a PowerShell path error when activating the virtual environment, and the pip install initially wrote to the system Python rather than the venv.	We standardised on python-oracledb in thin mode, which requires no Oracle client installation. A clear setup sequence in the README resolved environment differences quickly.
Database Compatibility Risk: Schema or connection issues between development environments	The TO_CHAR() date format strings in SQL caused an ORA-01745 error because python-oracledb scans for bind variable colons even inside string literals.	We removed all date formatting from SQL and let Python's datetime.strftime() handle formatting instead. This eliminated the error and actually simplified the code.
Integration Complexity Risk: Frontend and backend communication issues during development	The sessions dropdown did not populate initially because the seed data had not been re-run after the tables were created under the correct schema owner.	We tested each API endpoint directly in the browser before connecting the frontend, which made it straightforward to isolate and fix the data issue.
Time Constraint Risk: Limited time near the deadline could reduce testing and documentation quality	Losing a group member mid-project reduced available hours and required redistribution of tasks, particularly database setup and seed data responsibilities.	We focused on delivering a working core demonstration first: schema, one endpoint, one route on the map, before adding additional users and sessions. This kept the project on track.

API Dependency Risk: Google Maps usage limits or key misconfiguration could break map functionality	The Maps JavaScript API [2] loaded correctly once the API key was restricted to localhost:5000. No quota issues were hit during development.	We restricted the API key to our local development URL immediately after creation. This prevented unauthorised use and kept us within the free tier limits.

The most important takeaway from the risk exercise was that the risks we ranked as most likely, environment configuration and integration complexity, were actually the ones that took the longest to handle. Writing the risks in the proposal encouraged the group to consider the design's flaws before creating it, resulting in a preliminary strategy for each eventuality when it occurred.

2. Overall Experience

What went well

The database design came together cleanly. Choosing four clearly separated tables: users, sessions, waypoints, and routes, made the SQL straightforward and kept the relationships easy to reason about. The decision to add a composite index on GPS_WAYPOINTS(session_id, recorded_at) paid off immediately during testing, and being able to demonstrate it with EXPLAIN PLAN gave the project a strong database justification rather than just a working demo.

The Google Maps integration was smoother than expected. Once the API key was correctly configured and the waypoints were returned as an ordered JSON array, drawing the polyline on the map required only a few lines of JavaScript. The animation feature, moving a marker along the route in real time, was a late addition that turned out to be the most visually compelling part of the demonstration.

What was harder than expected

The python-oracledb bind variable issue was the most unexpected technical obstacle. The driver scanning for colon characters inside SQL string literals is not a behavior that most documentation warns about clearly, and it took several iterations to identify the root cause. The fix, removing all date formatting from SQL and handling it in Python, was simple once the problem was understood, but finding it cost time that had been budgeted for other work.

Managing the project with one fewer group member than planned added pressure to the final weeks. The redistribution of tasks worked out, but it reinforced how important it is to build the core demonstration first and treat additional features as optional additions rather than requirements.

What the group would do differently

- **On day one, test the database connection and the first API endpoint** before writing any frontend code. Having a working end-to-end path early would have caught the ORA-01745 error much sooner.
- **Keep a shared troubleshooting log during development.** Other group members later solved several issues individually. A shared document of errors and fixes would have saved time.
- **Run the seed data script in smaller batches during initial testing** rather than inserting everything at once to make it easier to spot data issues early.

Closing reflection

The project succeeded in its primary goal: a live, working demonstration of GPS location mapping backed by an Oracle 26ai [1] database. Every component, the schema, the API, the map, and the animation, functions as described in the proposal. More importantly, the group now has a concrete reference for how a full-stack database application is assembled from the ground up, which is a more durable takeaway than any individual course concept.

References

[1] Oracle Corporation. (2025). “Get Started with Oracle Database 26ai | Oracle Canada,” *Oracle.com*, 2026. https://www.oracle.com/ca-en/database/free/get-started/ (accessed Mar. 29, 2026).

[2] Google LLC. (2025). *Maps JavaScript API documentation*. Google for Developers. https://developers.google.com/maps/documentation/javascript (accessed Mar. 30, 2026).

[3] Zandbergen, P. A. (2009). Accuracy of iPhone locations: A comparison of assisted GPS, WiFi and cellular positioning. *Transactions in GIS, 13*(s1), 5–25. https://doi.org/10.1111/j.1467-9671.2009.01152.x (accessed Mar. 30, 2026).

[4] Grinberg, M. (2018). *Flask web development: Developing web applications with Python* (2nd ed.). O'Reilly Media. Flask Web Development, 2nd Edition [Book] (accessed Mar. 31, 2026).

[5] Office of the Privacy Commissioner of Canada. (2024). *Privacy in the workplace: Location tracking*. Government of Canada. https://www.priv.gc.ca/en/ (accessed Mar. 31, 2026).